Progetto di Object Orientation 2024-2025 Traccia 2 (hackathon)

Elaborato sviluppato da Davide Pagliaro e FrancescoPio Pellegrino

Indice

Considerazioni sul modello di dominio del problema, sulle classi individuate/scelte e su particolari scelte progettuali iniziali.....2
Breve descrizione del problema posto.....2
Classi individuate (CRC cards).....3
Dettagli organizzativi di progetto.....5
Standard di documentazione.....5
Scelte progettuali effettuate.....5
UML Class Diagram per il dominio del problema.....7
Presentazione del modello di dominio della soluzione.....8
Considerazioni sulla struttura generale dell'applicativo.....8
Dettagli software/tool utilizzati.....10
Dettagli implementativi.....11
Naming convention.....11
Pattern BCE.....11
Integrazione Data Access Object (DAO).....12
Struttura implementativa.....13
Considerazioni finali di progetto.....14
Possibili migliorie individuate.....14

Considerazioni sul modello di dominio del problema, sulle classi individuate/scelte e su particolari scelte progettuali iniziali

Breve descrizione del problema posto

Il problema richiede la creazione di un software che possa essere utilizzato per gestire la creazione e l'organizzazione di eventi hackathon.
L'hackathon prevede la produzione di software utile alla risoluzione di un dato problema.
Il software viene sviluppato da gruppi di partecipanti, ossia team.
Ogni partecipante ha la possibilità di creare o prendere parte ad un team purché il team o l'hackathon stesso non sia pieno.
L'hackathon viene creato da un organizzatore.
L'organizzatore specifica: data di fine e di inizio dell hackathon, numero massimo di team, numero massimo di partecipanti, data di inizio e di fine delle iscrizioni all hackathon.
L'organizzatore ha inoltre la facoltà di scegliere i giudici, coloro che valutano i progetti intermedi/finali (Dando consigli ed opinioni nel caso delle fasi intermedie dello sviluppo…) prodotti dai team e che presentano il problema da risolvere durante il corso dell hackathon.
Organizzatori, partecipanti e giudici sono tutti utenti della piattaforma tramite cui sono creati e organizzati gli hackathon.

Ogni hackathon ha luogo in una sede fisica.
Alla fine di ogni hackathon, viene presentata una classifica a punteggio, basata sui voti dati dai giudici ai progetti dei team, per determinare il vincitore.

Classi individuate (CRC cards)

Hackathon	
	Team, Location, Judge, Organizer, Participant, User

Team	
	Hackathon, Participant, Judge, Update,

Organizer	
Crea hackathon, apre le iscrizioni per uno o più hackathon, chiude le iscrizioni per uno o più hackathon, da inizio all hackathon, termina l'hackathon, invita i giudici per l'hackathon, pubblica la classifica con punteggio dell hackathon	Hackathon, Judge, User

User	
Può iscriversi all hackathon, può classificarsi come giudice per eventuali hackathon	Hackathon

Participant	
Può creare un team per l'hackathon, accetta/rifiuta un invito di un team.	Team, Participant, Leader, Hackathon

Leader	
Invita altri partecipanti ad entrare nel suo team, pubblica gli aggiornamenti riguardanti il progetto del suo team durante l'hackathon, visualizza la classifica finale dell hackathon	Team, Participant, Hackathon

Judge	
Può accettare/rifiutare inviti ad essere giudice per uno o più hackathon, Può pubblicare la descrizione del problema, Può pubblicare un'opinione su uno o più aggiornamenti di progetto, Può assegnare un voto ai progetti finali presentati per l'hackathon	Team, Hackathon, Update, Organizer, Opinion

Location	
	Hackathon

Update	
	Team, Opinion

Dettagli organizzativi di progetto

Standard di documentazione

Per la documentazione interna del progetto (In particolare per la parte di progettazione grafica) si è deciso di utilizzare lo standard UML (Unified Modeling Language) poiché in linea con i concetti rappresentati e descritti.
Il progetto prevede l'utilizzo di diversi componenti UML (package diagrams, sequence diagrams e class diagrams).
Ai componenti UML forniti sono affiancate considerazioni e specifiche testuali, presenti nel documento.

Scelte progettuali effettuate

A seguito di un'analisi del problema, sono state effettuate le seguenti scelte (per cui si espongono anche le motivazioni):

-**Ogni utente può proporsi come giudice, così da essere invitato per eventuali hackathon** (Si ritiene adeguato che l'organizzatore possa invitare solo utenti interessati a fare da giudici)

-**Ogni utente può iscriversi/partecipare ad un solo hackathon o creare un solo team per un solo hackathon per volta** (Questo perché gli hackathon si svolgono in loco, durante il periodo specificato dall'organizzatore, e perché se un utente si iscrivesse a più hackathon, l'organizzatore potrebbe far terminare le iscrizioni e dare inizio all hackathon contemporaneamente all hackathon già in corso).

-Un hackathon, alla creazione, non ha giudici (Organizzatori e giudici possono esistere anche senza occuparsi di un hackathon)

-Un organizzatore ha la possibilità di gestire più hackathon in un dato periodo.

-I giudici possono occuparsi di più hackathon in un dato periodo (Loro possono valutare i progetti tramite la piattaforma, non è necessario che siano presenti nella sede in cui ha luogo l'hackathon).

-I giudici possono dare ai progetti dei team un voto da 0 a 10

-Il punteggio di fine hackathon di un team è dato dalla somma dei voti forniti dai giudici (e.g. supponendo 4 giudici, se G1 vota 1, G2 vota 3, G3 vota 8, G4 vota 10, il punteggio assegnato al team sarà 22).

-Un utente iscritto ad un hackathon che non entra in un team entro l'inizio dell'evento (Ossia la chiusura delle iscrizioni), viene rimosso dai partecipanti non appena l'hackathon ha inizio.

-L'organizzatore chiude arbitrariamente le iscrizioni

-Il leader del team in un hackathon pubblica gli aggiornamenti sul progetto del team (Si ritiene opportuno considerare il capo team come

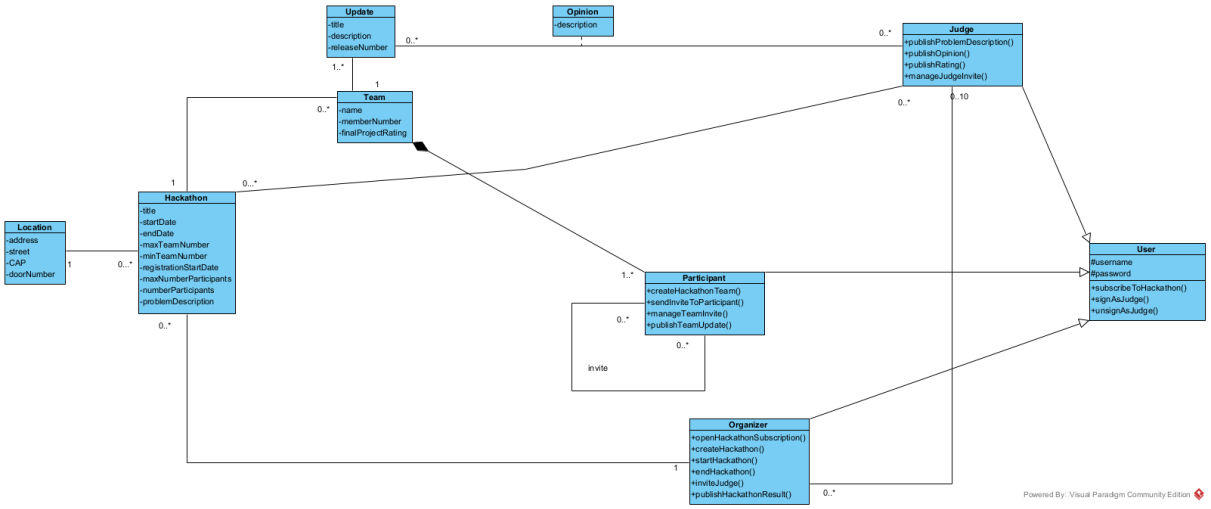
l’unico a poter confermare la validità di un aggiornamento, dunque pubblicarlo).

-Il leader del team è l’unico a poter visualizzare la classifica di fine hackathon

-Una volta iscritti, i partecipanti non possono abbandonare l’hackathon
-Gli utenti della piattaforma possono classificarsi come giudici
(L’organizzatore potrà poi invitarli a partecipare come giudici per l’hackathon)

-Un hackathon può iniziare senza giudici o senza partecipanti/team
(L’organizzatore sceglierai poi se chiuderlo o meno).

UML Class Diagram per il dominio del problema



Presentazione del modello di dominio della soluzione

Considerazioni sulla struttura generale dell’applicativo

Il software è strutturato utilizzando diverse pagine, includenti vari componenti interattivi.
Ogni pagina è destinata ad essere utilizzata dall’utente per un’azione particolare.
Le pagine disponibili sono:

-Home:

Si tratta della pagina centrale.
E’ una pagina adattiva, cambia rispetto al ruolo ricoperto dall’utente (Utente semplice, partecipante ad hackathon, leader per team di hackathon, giudice od organizzatore).

-Per gli utenti:

Viene mostrato un resoconto degli Hackathon con iscrizioni attive e un bottone per segnarsi come giudice.

-Per i partecipanti/leader:

Viene mostrato un resoconto dell hackathon corrente.
(Per i leader esiste la possibilità di caricare degli update tramite un’apposita pagina e di visionare i risultati dell hackathon a fine hackathon).

-Per i giudici:

Viene mostrato un resoconto degli hackathon gestiti
e degli inviti ricevuti da parte di organizzatori di hackathon.
Permette l’accesso alla pagina **“JudgeHackathonManager”**

tramite il resoconto degli hackathon gestiti (se presenti).
Viene inoltre mostrato (Qualora il giudice non sia impegnato nel gestire hackathon) un tasto che permette di rimuoversi come giudice dalla piattaforma.

-Per gli organizzatori:

Viene mostrato un resoconto degli hackathon correntemente gestiti.
Permette l'accesso alle pagine **"OrganizerHackathonManager"** e **"OrganizerHackathonCreate"** tramite il resoconto degli hackathon gestiti (se presenti).

-Login:

Contiene il form di login, dove vengono inserite le credenziali di accesso (username e password) dagli utenti.
Ha un bottone che porta alla finestra "Register" ed uno di "Sign in" che permette all'utente di accedere alla piattaforma (Ossia alla pagina Home).

-Register:

Contiene il form di registrazione, dove vengono inserite le credenziali (username e password) per la creazione di un profilo utente.

-JudgeHackathonManager:

Accessibile dalla home per giudici quando questo è impegnato per un hackathon.
Permette di:
-visualizzare gli update più recenti pubblicati dai team.
-dare opinioni agli update più recenti pubblicati dai team.
-pubblicare la descrizione del problema affrontato durante l'hackathon
-dare un voto ad un team (da 0 a 10) partecipante all hackathon

-OrganizerHackathonCreate

Accessibile dalla home per gli organizzatori.
Permette di:
-creare hackathon

-OrganizerHackathonManager

Accessibile della home per gli organizzatori quando questo è impegnato con hackathon creati.

Permette di:
-invitare giudici per l hackathon
-chiudere le iscrizioni all hackathon
-terminare l'hackathon

Dettagli software/tool utilizzati

Il software è sviluppato utilizzando il linguaggio di programmazione orientato agli oggetti **Java**.
Questa scelta permette di avere un **software portabile** e **facilita lo sviluppo del software stesso grazie all'utilizzo di package testati e consolidati forniti dal linguaggio**.
In particolare, il software prevede anche un' interfaccia GUI che ne semplifichi l'utilizzo per l'utente finale.
Java fornisce le **librerie di creazione GUI "Swing" e "AWT"** che, oltre ad essere affermate, **sono di semplice utilizzo** (Risultano quindi perfette per quello che è richiesto).
Per semplificare ulteriormente la creazione del software, **si utilizza l'IDE "IntelliJ Idea"** fornito da JetBrains che **include diversi plugin utili in fase di sviluppo e post-sviluppo**.

Nel caso considerato, si parla dei seguenti plugin:
-Swing UI Designer: Permette una più semplice gestione/creazione della GUI creata utilizzando la libreria Swing, in particolare, gestisce in maniera automatica il posizionamento dei vari widget e panel all'interno dei frame generati, seguendo layout predefiniti tra cui lo sviluppatore può scegliere.
-JavaDoc: Permette la generazione di javadocs per il codice fornito.
-Maven: Semplifica lo sviluppo generale del progetto (In particolare, tramite la build automation)

Dettagli implementativi

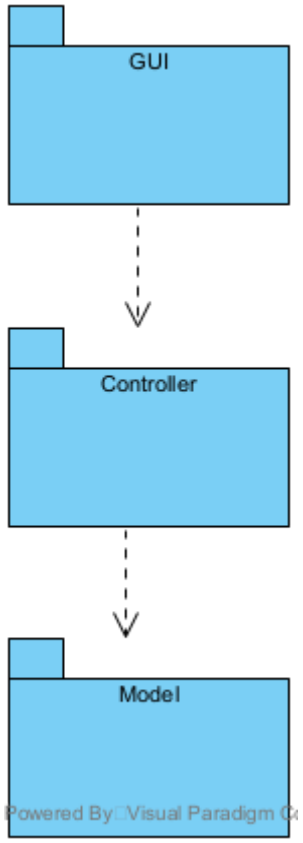
Naming convention

Per il progetto, si utilizza come naming convention la **camelcase notation**, in linea con lo standard Java.
Questa convenzione è applicata ad oggetti, variabili e metodi (Unica **eccezione fatta per metodi costruttori, il cui nome inizia sempre con una lettera maiuscola**).

Pattern BCE

Il progetto è implementato in linea con il pattern BCE (Boundary-Control-Entity) che prevede la presenza di tre principali package:
-Boundary: È il package contenente i componenti della GUI
-Controller: È il package contenente la classe “Controller” del controller
-Entity: È il package contenente le classi incluse nel domain model
Seguendo il modello BCE:
-GUI prende ciò di cui ha bisogno dal controller.
-Controller fornisce dati a GUI, prendendoli dal domain model
-Domain model fornisce dati a Controller.

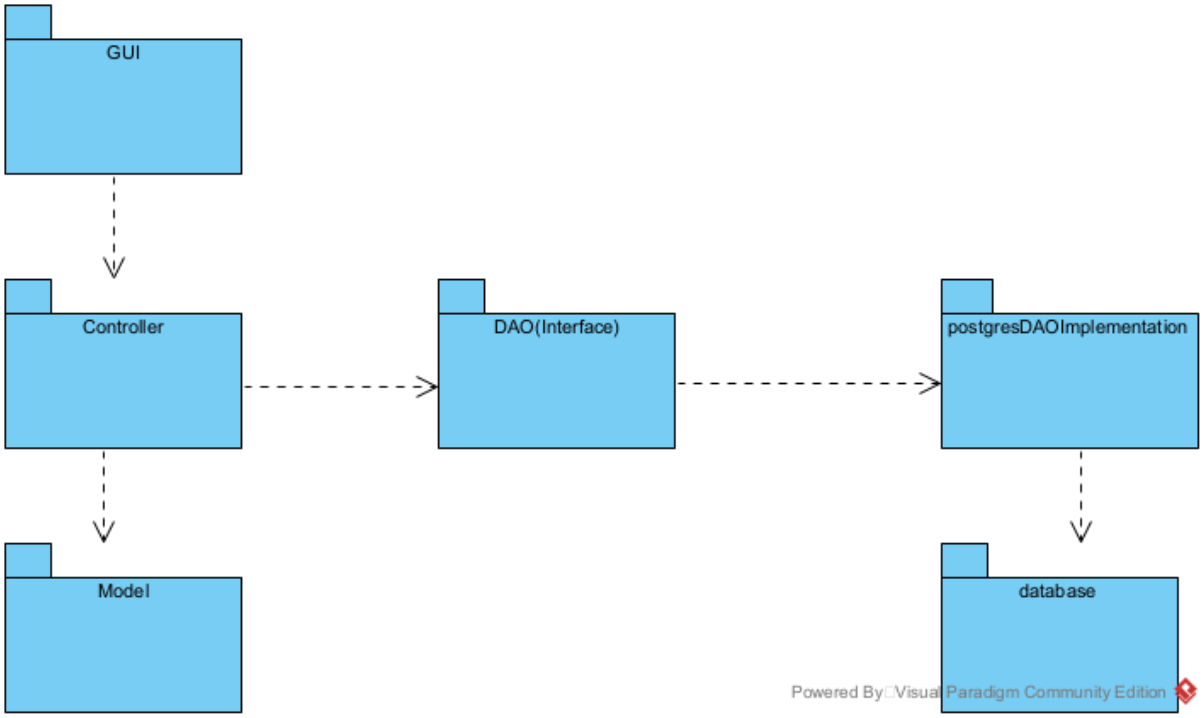
Si ha quindi il **seguente package diagram** (Per cui si forniscono assieme alla documentazione i relativi class Diagrams):



Integrazione Data Access Object (DAO)

In modo da permettere la persistenza dei dati alla chiusura dell'applicativo, al pattern BCE è **integrato il pattern DAO** (Data Access Object) che prevede la **creazione di interfacce generiche di accesso ad un database** ed **una loro implementazione non generica** che permette il retrieve dei dati.
Nel caso particolare, si utilizza come DBMS postgreSQL (Quindi un database relazionale).

Sia interfacce che implementazioni sono contenute in appositi package java.
Ad interfacce e relative implementazioni si aggiunge un **package database** in cui, per comodità, è **incluso un metodo statico che permette l'ottenimento di istanze di connessione al database**.
Le istanze di connessione vengono usate dalle implementazioni per connettersi al database e compiere operazioni CRUD.
Si presenta il **package diagram** ottenuto a seguito delle modifiche appena menzionate (Pattern BCE + DAO).



Struttura implementativa

L'applicativo è strutturato nel seguente modo:

1) L'utente avvia l'applicativo e si ritrova nella pagina di Login(package GUI).
Il costruttore Login() crea un'istanza di "Controller" (Package controller).
Il costruttore **Controller() inizializza le strutture dati necessarie a mantenere i dati transienti** poi, le **popola chiamando il metodo "retrieveDataAndInitialize()"**, presente sempre nella classe controller, che effettua il retrieve dei dati dal database, richiamando gli appositi metodi di retrieve (Package postgresDAOImplementation, classe "DAORetriever").

2)Quando il retrieve è stato effettuato, l'utente può usare la piattaforma.
Interagisce con i dati transienti, effettuando azioni di update/delete/insert sul database quando necessario (Package postgresDAOImplementation, classe "DAOUpdater").

Considerazioni finali di progetto

Possibili migliorie individuate

1) Come attualmente implementato, i**l retrieve iniziale dei dati esegue molte interrogazione al database separate** tra loro.
Si potrebbe decidere di ridurre il numero di accessi al database cercando di **unire i retrieve che sono correlati** (Le foreign key non vengono sfruttate al meglio).

2) Alcuni riferimenti tra classi potrebbero essere rimossi (**Sono presenti ridondanze dei dati**, come per esempio il campo “currentHackathon” in “Location”)

3) La gestione delle sedi per gli hackathon non è effettuata nel migliore dei modi(**Ogni organizzatore crea una nuova sede per hackathon, poi, la elimina**).
Si potrebbe tenere traccia di alcune sedi che possano essere occupate per gli hackathon (Ma non create ogni volta).

4) **Il caso in cui un utente ignori un invito** (Che sia questo un giudice o un partecipante ad hackathon) e le registrazioni all hackathon o l hackathon stesso siano chiusi non è gestito completamente (Gli inviti non vengono eliminati quando ignorati).

5) Si suppone che gli organizzatori siano già registrati alla piattaforma (**Hard coding degli organizzatori**), non è previsto un meccanismo di creazione degli organizzatori.

6) Alcuni metodi del model sono stati implementati ma non utilizzati (e.g. quelli di gestione degli inviti inclusi nella classe “Judge” e nella classe “Participant” del package model).